

5 Network Algorithms

We have seen two main ways to represent a network, as a list of vertices and edges or as an adjacency matrix. In computer calculations we want to use a representation that optimises the running time of our algorithms while still fitting in RAM. For this reason storing the full adjacency matrix is usually quite inefficient and often impossible, since a network with large N would require storing N^2 entries, we need more efficient methods.

Reading

[1] **Chapters 9, 11.**

5.1 Sparse Matrices

Real networks are usually far from fully connected, so most of the entries in the adjacency matrix are zero. If the adjacency matrix is mostly zeros it is often represented as a **sparse matrix**. There are several ways to define these, with their own disadvantages and advantages. In a sparse matrix we save all the non-zero entries and assume anything not saved is zero. The simplest data structure implementing this would be a dictionary or hashmap using tuples (i, j) as keys.

$$d[(i, j)] = 1$$

Alternatively the pair could be turned into an index $k = Ni + j$ instead of a tuple. We often want to do matrix multiplications with sparse matrices, in which case the dictionary is not very efficient. A popular alternative is a **coordinate list**, often called COO format. Which is just a list of tuples of the form

$$(\text{row}, \text{column}, \text{value})$$

However, probably the most common method you will see is **compressed sparse row** (CSR) format. This stores the values in three lists

$$\text{values}, \text{column}, \text{row}$$

The values and column arrays work as you probably expect but row entries do not! They are defined by

$$\text{row}[0] = 0$$

$$\text{row}[i] = \text{row}[i - 1] + \text{number of non-zero elements in the } (i - 1)^{\text{th}} \text{ row of the matrix.}$$

For example the matrix

$$A = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 3 & 0 & 4 \\ 0 & 0 & 0 & 5 \end{pmatrix}$$

Would be

$$\begin{aligned} \text{values} &= [1, 2, 3, 4, 5] \\ \text{col} &= [0, 3, 1, 3, 3] \\ \text{row} &= [0, 2, 4, 5] \end{aligned}$$

To get the nonzero values in row r we compute

$$r_{start} = \text{row}[r], \quad r_{end} = \text{row}[r + 1]$$

and use these secondary indices to slice the values and columns lists

For example, row 1 of the matrix above gives

$$(r_{start}, r_{end}) = (2, 4)$$

The values in row 1 are then `values[2:4] = [3,4]` at positions `col[2:4] = [1,3]`. This is good to know, but generally you will use some existing implementation and not worry about it!

5.2 Network data structures

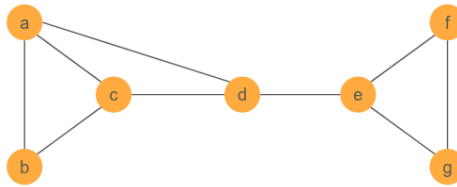


Figure 1: A network to represent in code.

For storing the network, rather than the adjacency matrix, the most common way is as an **adjacency list**. The table below gives the adjacency list for the network in Figure 1.

a	b,c,d
b	a,c
c	a,b,d
d	a,c,e
e	d,f,g
f	e,g
g	e,f

Usually this is fine for most purposes, but depending on the types of algorithms we plan to run and networks we have to study, we may want to optimise certain operations. For example, we may have networks where the average degree is high and algorithms that require checking if some node j is a neighbour of another node i . In other applications we might want to frequently add or delete nodes and edges.

One way to optimise for different cases is to use different structures instead of a list to store the neighbours. For example, if we store the neighbours in a tree data structure we have an *adjacency tree*. If we store them in a hash map we have an *adjacency map*. Depending on the average number of look ups and the speed of the tree or hash map implementation, either of these methods might be faster or slower than a simple list and the best approach is application dependent.

We can also just store the edge list. The edge list for the network in Figure 1 is

a	b
a	c
a	d
b	c
c	d
d	e
e	f
e	g
f	g

For undirected networks we assume an edge (a, b) implies there is an edge (b, a) . Network data stored in various repositories is often saved in this format. For use in algorithms the edge list is not typically used, because to find a particular node we have to search from the beginning however, there is a way to make this much more efficient - by assuming the nodes are labelled by integers and storing the cumulative degrees of all the vertices in

a separate list.

If we make the obvious relabelling $a \rightarrow 0, b \rightarrow 1, \dots$, the degree sequence is

$$k = (3, 2, 3, 3, 3, 2, 3)$$

and the cumulative degree sequence is defined by the recursion

$$K_0 = k_0$$

$$K_i = K_{i-1} + k_i$$

$$K = (3, 5, 8, 11, 14, 16, 19)$$

We also condense the edge list to

$$l = (1, 2, 3, 0, 2, 0, 2, 3, 0, 2, 4, 3, 5, 6, 4, 6, 4, 6)$$

which is just

$$l = (\text{neighbours of } 0, \text{neighbours of } 1, \dots)$$

Say we want to find the neighbours of node 4. We know it has

$$k_4 = K_4 - K_3 = 14 - 11 = 3$$

neighbours and these are

$$l[K_4 : K_4 + k_i]$$

In this case

$$l[11 : 14] = (l_{11}, l_{12}, l_{13}) = (3, 5, 6) = (d, f, g)$$

5.3 Force directed layout

While really big networks with millions of nodes are hard to visualise, often we have a manageable number of nodes say between 2 and a couple of thousand. In this case network visualisations can be very insightful. The most common way to do this is to use a **force directed layout**, in particular a **spring system**. The basic idea is to think of the nodes as electrically charged balls and the edges as springs. These springs are not quite like real springs, they attract each other with a force

$$c_1 \log(d/c_2)$$

where d is the distance between nodes and c_1, c_2 are constants. Note d is not the graph distance, here we give the nodes a position in some 2d plane x_i and $d_{ij} = \|x_i - x_j\|$. The nodes repel each other with the force

$$c_3/d^2$$

where c_3 is another constant. Starting from some random positions the nodes are moved by a small amount proportional to the total force acting on them (remember, force is a vector). This is a variation of the **Fruchterman–Reingold algorithm**. There are many refinements to this which make the outputs ‘nicer’ (by some definition) but they often build on or refine this basic idea.

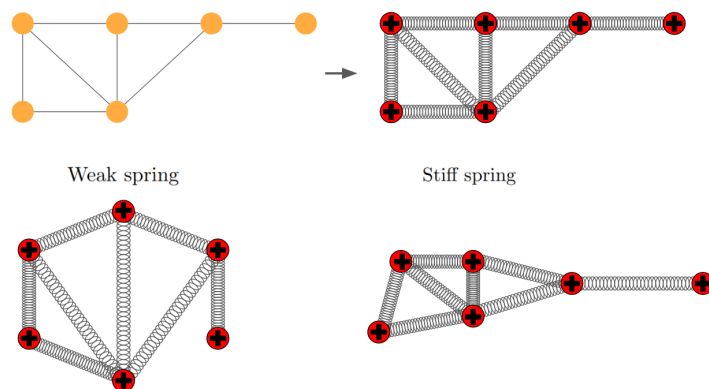


Figure 2: The network in the top left is transformed into the spring and charge system on the right and is allowed to ‘relax’. For weak springs the charges will repel and try to stay far apart from each other and end up in a circle. For strong springs, clusters of highly connected nodes condense and loosely connected nodes are repelled from these clusters of charge.

5.4 Spectral methods

We have already discussed eigenvector centrality and related measures. These work with the adjacency matrix. There is a matrix called the **graph Laplacian** which turns out to be the more natural matrix to consider when looking at transport or flows on a network. Methods related to eigenvectors of the Laplacian are called **spectral methods** (nothing to do with ghosts, the eigenvalues of a matrix are often referred to as its spectrum for historical reasons).

To motivate it, consider some quantity ψ that has the value ψ_i on each of the nodes. Suppose that the quantity flows along the network from i to

its neighbours j at a rate $C(\psi_i - \psi_j)$, C is called the diffusion constant. This means that the ‘stuff’ ψ flows from nodes where there is more of it to nodes where there is less. The rate of change of ψ_i comes from adding up all the contributions to and from the neighbours so

$$\frac{d\psi_i}{dt} = C \sum_j A_{ij}(\psi_j - \psi_i)$$

You can think of this as adding stuff flowing from j to i and subtracting stuff flowing from i to j . With some algebra the RHS becomes

$$\frac{d\psi_i}{dt} = C \sum_j (A_{ij} - \delta_{ij}k_i)\psi_j$$

δ_{ij} is the Kronecker delta. In matrix-vector form

$$\frac{d\psi}{dt} = C(A - D)\psi \quad (1)$$

where D is a matrix with the degrees on the diagonal and 0s elsewhere. It is common to rewrite this as

$$\frac{d\psi}{dt} + CL\psi = 0$$

where

$$L = D - A \quad (2)$$

defines the graph Laplacian.

If we know the eigenvalues and eigenvectors of L

$$Lv_i = \lambda_i v_i$$

Then we can express any vector as a sum, $\psi(t) = \sum_i c_i(t)v_i$ and substitute this into the equation,

$$\sum_i \left(\frac{dc_i}{dt} + C\lambda_i c_i \right) v_i = 0$$

Implying that

$$\frac{dc_i}{dt} + C\lambda_i c_i = 0 \implies c_i(t) = c_i(0)e^{-C\lambda_i t}$$

So the eigenvectors and eigenvalues of L tell us about solutions to the diffusion equation on the network and therefore how stuff ‘flows’ on it. We

can also use eigenvectors of the Laplacian to solve **graph partitioning** problems which we will discuss in the next section.

The most important eigenvalues of a matrix are usually the largest or smallest ones. For the Laplacian usually we want the smallest ones, since these control the long term behaviour. It turns out to the smallest one is easy to find. Consider the vector $e = (1, 1, 1, \dots, 1)$

$$Le = Ae - De = \sum_j A_{ij} - \delta_{ij}k_i = k_i - k_i = 0$$

So e is an eigenvector of L with eigenvalue 0. Usually this is not useful and the second largest eigenvector, called the **Fiedler vector** is often the one we want.

We can find a large eigenvector using the power method. Any vector v can be expressed in the eigenbasis of L

$$v = \sum_j c_j v_j$$

repeatedly applying L gives

$$L^k v = \sum_j c_j \lambda_j^k v_j$$

For k large enough, $L^k v$ will converge to the largest eigenvector. But we want the smallest one. This is harder but one way to compute it is through **inverse iteration**. Choose a vector

$$v \perp e \implies v = c_1 v_1 + c_2 v_2 + \dots$$

i.e. there is no component of the 0 eigenvalue. Now we iterate

$$L^{-k} v = \sum_j c_j \lambda_j^{-k} v_j$$

As with the power method, this will eventually converge to the smallest eigenvector present in v , namely v_1 , the Fiedler vector. This is much slower than power iteration because at every step we have to apply the inverse of L which requires repeatedly solving

$$Lx = (L^{k-1}v)$$

where the RHS is known from the previous iteration. Because of numerical issues, it is also necessary to orthogonalise the vector $L^k v$ after every step.

$$L^k v \rightarrow L^k v - \frac{L^k v \cdot e}{N} e$$

There are more advanced approaches, but in general computing the Fiedler vector is difficult.

5.5 Kernighan–Lin algorithm

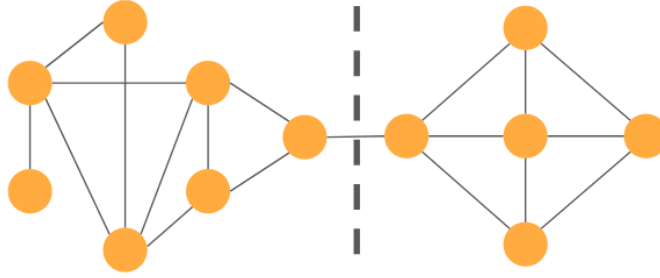


Figure 3: Network with a bridge.

The most basic form of the network partitioning problem is to divide the network into two groups. We might have a situation like Figure 3 where we have an ‘almost’ two component network. In graph theory an edge like this, which when cut divides the network into two components is called a **bridge**. We would like an algorithm to identify situations like this, that is to partition of the network into two parts such that the number of edges between the parts is minimised. The number of edges between parts is called the **cut size**, so this is sometimes called the **min cut** problem.

Kerningham and Lin (KL) came up with a heuristic algorithm to solve this problem, since an exact solution involves checking an infeasibly enormous number of cases. The algorithm is as follows

1. Pick two sizes for the parts N_1 and N_2 where $N_1 + N_2 = N$. This is an input to the algorithm.
2. Assign each vertex to a group in some way e.g. randomly.

3. For nodes i in one part and j in the other, find the pair that, if swapped, reduce the cut size the most or increase it the least and perform the swap.
4. Do this again but exclude from consideration any nodes which have already been swapped.
5. When it is no longer possible to swap nodes stop and go back to the state with the smallest cut size.
6. Repeat from step 3 until there is no improvement in the minimum cut.

As you can guess, this can be slow and has a number of other disadvantages e.g. how do we pick N_1 and N_2 ? In the next Chapter we will talk about more effective ways to partition a network.

References

- [1] Mark Newman. *Networks*. Oxford university press, 2018.